

Hướng dẫn quản lý biến môi trường trong Angular

Giới thiệu

Trong phát triển ứng dụng Angular, việc quản lý các biến môi trường là rất quan trọng để cấu hình ứng dụng cho các môi trường khác nhau như phát triển (development), thử nghiệm (staging) và sản xuất (production). Các biến môi trường có thể bao gồm API endpoint, khóa API, hoặc các cài đặt cấu hình khác nhau tùy thuộc vào môi trường triển khai.

Hướng dẫn này sẽ trình bày các phương pháp tốt nhất để quản lý biến môi trường trong Angular, bao gồm cấu hình cơ bản, các kỹ thuật nâng cao, và các cân nhắc về bảo mật.

1. Cấu hình cơ bản với `environment.ts`

Angular CLI cung cấp một cơ chế tích hợp sẵn để quản lý các biến môi trường thông qua các tệp `environment.ts`. Theo mặc định, bạn sẽ có hai tệp:

- `src/environments/environment.ts` : Dành cho môi trường phát triển.
- `src/environments/environment.prod.ts` : Dành cho môi trường sản xuất.

Cấu trúc tệp `environment.ts`

Mỗi tệp `environment.ts` xuất một đối tượng chứa các biến môi trường. Ví dụ:

```
// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api',
  apiKey: 'dev-api-key'
};
```

```
// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.your-app.com/api',
  apiKey: 'prod-api-key'
};
```

Sử dụng biến môi trường trong ứng dụng

Để sử dụng các biến này trong ứng dụng của bạn, bạn chỉ cần import đối tượng `environment`:

```
// app.component.ts
import { Component } from '@angular/core';
import { environment } from '../environments/environment';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Angular Environment Variables';
  apiUrl = environment.apiUrl;
  isProduction = environment.production;

  constructor() {
    console.log('API URL:', this.apiUrl);
    console.log('Is Production:', this.isProduction);
  }
}
```

Xây dựng ứng dụng với các môi trường khác nhau

Khi bạn xây dựng ứng dụng Angular, CLI sẽ tự động chọn tệp môi trường phù hợp dựa trên cấu hình build:

- Để xây dựng cho môi trường phát triển:

```
ng build
```

(Sử dụng `environment.ts`)

- Để xây dựng cho môi trường sản xuất:

```
ng build --configuration=production
```

Hoặc ngắn gọn hơn:

```
ng build --prod
```

(Sử dụng `environment.prod.ts`)

Angular CLI sẽ thay thế tệp `environment.ts` bằng `environment.prod.ts` (hoặc bất kỳ tệp môi trường nào được cấu hình) trong quá trình build. Điều này đảm bảo rằng các biến môi trường chính xác được nhúng vào bản dựng cuối cùng của ứng dụng.

2. Cấu hình môi trường tùy chỉnh

Bạn có thể định nghĩa thêm các môi trường tùy chỉnh (ví dụ: `staging` , `qa`) bằng cách tạo các tệp `environment.staging.ts` , `environment.qa.ts` , v.v. và cấu hình chúng trong tệp `angular.json` .

Bước 1: Tạo tệp môi trường mới

Ví dụ, tạo `src/environments/environment.staging.ts` :

```
// src/environments/environment.staging.ts
export const environment = {
  production: false,
  apiUrl: 'https://api.staging.your-app.com/api',
  apiKey: 'staging-api-key'
};
```

Bước 2: Cấu hình trong `angular.json`

Trong tệp `angular.json` , tìm đến phần `projects.[your-project-name].architect.build.configurations` và thêm cấu hình cho môi trường mới:

```

{
  ...
  "projects": {
    "your-app-name": {
      ...
      "architect": {
        "build": {
          "builder": "@angular-devkit/build-angular:browser",
          "options": {
            ...
          },
          "configurations": {
            "production": {
              "fileReplacements": [
                {
                  "replace": "src/environments/environment.ts",
                  "with": "src/environments/environment.prod.ts"
                }
              ],
              ...
            },
            "staging": {
              "fileReplacements": [
                {
                  "replace": "src/environments/environment.ts",
                  "with": "src/environments/environment.staging.ts"
                }
              ],
              "optimization": true,
              "outputHashing": "all",
              "sourceMap": false,
              "namedChunks": false,
              "extractLicenses": true,
              "vendorChunk": false,
              "buildOptimizer": true,
              // Thêm các cấu hình khác cho môi trường staging nếu cần
            }
          }
        },
        "serve": {
          "builder": "@angular-devkit/build-angular:dev-server",
          "options": {
            "browserTarget": "your-app-name:build"
          },
          "configurations": {
            "production": {

```

```
"browserTarget": "your-app-name:build:production"
    },
    "staging": {
      "browserTarget": "your-app-name:build:staging"
    }
  }
}
}
```

Bước 3: Xây dựng và phục vụ ứng dụng với môi trường tùy chỉnh

Để xây dựng hoặc phục vụ ứng dụng với môi trường staging :

```
ng build --configuration=staging
# Hoặc
ng serve --configuration=staging
```

3. Bảo mật biến môi trường và dữ liệu nhạy cảm

QUAN TRỌNG: Các biến môi trường được quản lý bằng cách thay thế tệp trong quá trình build sẽ được nhúng trực tiếp vào bundle JavaScript cuối cùng của ứng dụng. Điều này có nghĩa là bất kỳ ai cũng có thể xem các giá trị này bằng cách kiểm tra mã nguồn của ứng dụng trong trình duyệt. **Tuyệt đối không lưu trữ thông tin nhạy cảm (ví dụ: khóa API bí mật, thông tin đăng nhập cơ sở dữ liệu) trực tiếp trong các tệp `environment.ts` hoặc bất kỳ tệp nào được đưa vào bản dựng frontend.**

Các phương pháp bảo mật:

1. **Sử dụng Backend Proxy:** Đối với các khóa API hoặc thông tin nhạy cảm cần thiết cho các cuộc gọi API, hãy định tuyến các yêu cầu này thông qua một máy chủ backend (proxy). Máy chủ backend sẽ lưu trữ và sử dụng các khóa API nhạy cảm một cách an toàn, sau đó chuyển tiếp yêu cầu đến API bên ngoài. Ứng dụng Angular của bạn chỉ giao tiếp với backend của riêng bạn.

2. Biến môi trường thời gian chạy (Runtime Environment Variables):

Thay vì nhúng các biến môi trường vào thời gian build, bạn có thể tải chúng vào thời gian chạy. Điều này cho phép bạn sử dụng cùng một bản dựng ứng dụng cho nhiều môi trường mà không cần phải build lại.

Các phương pháp phổ biến bao gồm:

- **Tải từ tệp JSON:** Tải một tệp JSON chứa cấu hình môi trường từ máy chủ khi ứng dụng khởi động. Tệp này có thể được quản lý riêng biệt cho từng môi trường.
- **Sử dụng biến môi trường của máy chủ:** Nếu ứng dụng Angular được phục vụ bởi một máy chủ Node.js hoặc một dịch vụ backend khác, bạn có thể truyền các biến môi trường từ máy chủ xuống ứng dụng frontend thông qua một API endpoint hoặc bằng cách tạo một tệp cấu hình động.

3. Sử dụng các dịch vụ quản lý bí mật: Đối với các ứng dụng lớn và phức tạp, hãy sử dụng các dịch vụ quản lý bí mật chuyên dụng như AWS Secrets Manager, Azure Key Vault, hoặc HashiCorp Vault. Các dịch vụ này giúp lưu trữ và truy xuất các bí mật một cách an toàn, thường được tích hợp với CI/CD pipeline và các dịch vụ backend.

4. Best Practices

1. **Phân biệt rõ ràng giữa biến môi trường và cấu hình ứng dụng:** Biến môi trường nên được sử dụng cho các giá trị thay đổi giữa các môi trường triển khai (ví dụ: `apiUrl`). Cấu hình ứng dụng (ví dụ: `pageSize` mặc định, `featureFlags`) có thể được quản lý trong các tệp cấu hình riêng biệt hoặc trong chính mã nguồn nếu chúng không thay đổi theo môi trường.
2. **Sử dụng tiền tố cho biến môi trường:** Để tránh xung đột tên và dễ dàng nhận diện, hãy sử dụng tiền tố nhất quán cho các biến môi trường của bạn (ví dụ: `APP_API_URL`, `APP_FEATURE_TOGGLE`).
3. **Tài liệu hóa:** Luôn tài liệu hóa các biến môi trường mà ứng dụng của bạn sử dụng, mục đích của chúng và giá trị mong đợi cho từng môi trường. Điều này giúp các nhà phát triển khác dễ dàng hiểu và quản lý.
4. **Tránh hardcoding:** Không bao giờ hardcode các giá trị cấu hình hoặc API endpoint trực tiếp vào mã nguồn. Luôn sử dụng biến môi trường

hoặc các cơ chế cấu hình khác.

5. **Kiểm tra và xác thực:** Đảm bảo rằng các biến môi trường được tải và sử dụng đúng cách trong ứng dụng của bạn. Thực hiện kiểm tra tự động để xác minh rằng các cấu hình môi trường chính xác được áp dụng trong các bản dựng khác nhau.

6. **Sử dụng các thư viện hỗ trợ (nếu cần):** Đối với các trường hợp phức tạp hơn hoặc khi cần các tính năng nâng cao như biến môi trường thời gian chạy, bạn có thể xem xét các thư viện như `@ngx-env/builder` hoặc tự xây dựng một dịch vụ quản lý cấu hình tùy chỉnh.

- `@ngx-env/builder` : Một thư viện cho phép bạn sử dụng các biến môi trường từ hệ thống (ví dụ: từ tệp `.env` hoặc biến môi trường của hệ điều hành) trong ứng dụng Angular của bạn. Điều này hữu ích cho việc quản lý các biến môi trường mà không cần sửa đổi `angular.json` cho từng môi trường.

Kết luận

Quản lý biến môi trường hiệu quả là một phần quan trọng của việc phát triển và triển khai ứng dụng Angular. Bằng cách tuân thủ các phương pháp tốt nhất và hiểu rõ các cân nhắc về bảo mật, bạn có thể đảm bảo rằng ứng dụng của mình hoạt động chính xác và an toàn trên mọi môi trường.